

MODUL PRAKTIKUM  
REKAYASA PERANGKAT LUNAK



Nama : Tegar Faturrahman

NIM : 2403015027

Dosen Pengampu : Ir. Arry Avorizano, S.Kom., M.Kom.

Nama Proyek : Sawit : The Last Chance

PROGRAM STUDI TEKNIK INFORMATIKA  
FAKULTAS TEKNOLOGI INDUSTRI DAN INFORMATIKA  
UNIVERSITAS MUHAMMADIYAH PROF.DR HAMKA  
JAKARTA 2026

## Pertemuan 10 – Implementasi Perangkat Lunak (Coding)

### Implementasi Fitur Utama

Pada proyek game “*Sawit: The Last Chance*” yang dikembangkan menggunakan Roblox Studio dan bahasa pemrograman Lua, diimplementasikan beberapa fitur utama yang mendukung gameplay dan alur cerita permainan. Berikut merupakan dua fitur utama yang diimplementasikan dalam sistem.

#### Program Utama :

```
local Players = game:GetService("Players")
local TweenService = game:GetService("TweenService")
local UserInputService = game:GetService("UserInputService")
local StarterGui = game:GetService("StarterGui")
local SoundService = game:GetService("SoundService")
local RunService = game:GetService("RunService")

local player = Players.LocalPlayer
local camera = workspace.CurrentCamera

local screenGui = script:FindFirstAncestorWhichIsA("ScreenGui")

local mainFrame = screenGui:WaitForChild("MainFrame")
local popups = screenGui:FindFirstChild("Popups")
local transition = screenGui:WaitForChild("Transition")
local storyFrame = screenGui:WaitForChild("StoryFrame")

local menuFolder = workspace:WaitForChild("Menu")
local storyCameraFolder = menuFolder:WaitForChild("StoryCameras")

-- =====
-- SOUND SETUP
-- =====

local menuSounds = SoundService:WaitForChild("MenuSounds")
local menuMusic = menuSounds:WaitForChild("MenuMusic")
local storyMusic = menuSounds:WaitForChild("StoryMusic")

local MENU_VOLUME = 0.8
local STORY_VOLUME = 0.65

local MENU_LOOP_START = 7
local MENU_LOOP_END = 38

local STORY_START_TIME = 178

local menuLoopConnection = nil

local playButton
```

```

if script.Parent:IsA("GuiButton") then
    playButton = script.Parent
else
    playButton = script.Parent:FindFirstChildWhichIsA("GuiButton", true)
end

if not playButton then
    warn("Play button tidak ketemu. Pastikan object Play punya
    TextButton/ImageButton di dalamnya.")
    return
end

local dialogueBox = storyFrame:FindFirstChild("DialogueBox")

local speakerLabel
local storyText
local hint

if dialogueBox then
    speakerLabel = dialogueBox:WaitForChild("Speaker")
    storyText = dialogueBox:WaitForChild("StoryText")
    hint = dialogueBox:WaitForChild("Hint")
else
    speakerLabel = storyFrame:WaitForChild("Speaker")
    storyText = storyFrame:WaitForChild("StoryText")
    hint = storyFrame:WaitForChild("Hint")
end

local STORY = {
    {
        Speaker = "Narator",
        Text = "Kamu adalah warga desa setempat yang merupakan pegawai di
        pabrik sawit, dan tugasmu adalah memanen sawit dan menjualnya pada pabrik sawit",
        Camera = "StoryCam1"
    },
    {
        Speaker = "Narator",
        Text = "Awalnya, desa ini terlihat tenang. Warga bekerja seperti biasa,
        sungai mengalir, dan perkebunan menjadi bagian dari kehidupan mereka.",
        Camera = "StoryCam2"
    },
    {
        Speaker = "Narator",
        Text = "Namun, setelah pabrik ini dibangun, semuanya berubah. Pabrik
        ini, mengeluarkan debu dan suara yang keras. Sungai mulai berair kotor.",
        Camera = "StoryCam3"
    },
}

```

```

        Speaker = "Narator",
        Text = "saat kamu bekerja di pabrik, kamu merasa seperti ada yang akan
dilakukan oleh bos pabrik yang akan berdampak terhadap lingkungan hidup desa ini.",
        Camera = "StoryCam4"
    },
    {
        Speaker = "Narator",
        Text = "Ini mungkin kesempatan terakhir desa ini sebelum bencana
benar-benar terjadi.",
        Camera = "StoryCam4"
    }
}

```

```

local isRunning = false
local controls = nil

```

```

local function getControls()
    if controls then
        return controls
    end

    local success, result = pcall(function()
        local playerModule =
require(player:WaitForChild("PlayerScripts"):WaitForChild("PlayerModule"))
        return playerModule:GetControls()
    end)

    if success then
        controls = result
    else
        warn("PlayerModule controls gagal diambil.")
    end

    return controls
end

```

```

local function setCoreGuiEnabled(enabled)
    pcall(function()
        StarterGui:SetCoreGuiEnabled(Enum.CoreGuiType.All, enabled)
    end)
end

```

```

local function setPlayerControl(enabled)
    local playerControls = getControls()

    if playerControls then
        if enabled then
            playerControls:Enable()
        else

```

```

        playerControls:Disable()
    end
end

local character = player.Character or player.CharacterAdded:Wait()
local humanoid = character:FindFirstChildOfClass("Humanoid")
local root = character:FindFirstChild("HumanoidRootPart")

if humanoid then
    humanoid.WalkSpeed = enabled and 16 or 0
    humanoid.JumpPower = enabled and 50 or 0
    humanoid.JumpHeight = enabled and 7.2 or 0
    humanoid.AutoRotate = enabled
end

if root then
    root.Anchored = false
end
end

local function tween(object, properties, duration)
    if not object then
        return
    end

    local tweenInfo = TweenInfo.new(
        duration or 0.5,
        Enum.EasingStyle.Quad,
        Enum.EasingDirection.InOut
    )

    local createdTween = TweenService.Create(object, tweenInfo, properties)
    createdTween:Play()
    createdTween.Completed:Wait()
end

local function fadeSound(sound, targetVolume, duration, stopAfterFade)
    if not sound then
        return
    end

    if not sound.IsPlaying then
        sound:Play()
    end

    local soundTween = TweenService.Create(
        sound,
        TweenInfo.new(duration or 1, Enum.EasingStyle.Quad,
Enum.EasingDirection.InOut),

```

```

        {
            Volume = targetVolume
        }
    )

    soundTween:Play()
    soundTween.Completed:Wait()

    if stopAfterFade then
        sound:Stop()
    end
end

local function startMenuMusic()
    menuMusic.Volume = MENU_VOLUME
    menuMusic.Looped = false
    menuMusic.TimePosition = MENU_LOOP_START

    if not menuMusic.IsPlaying then
        menuMusic:Play()
    end

    if menuLoopConnection then
        menuLoopConnection:Disconnect()
        menuLoopConnection = nil
    end

    menuLoopConnection = RunService.Heartbeat:Connect(function()
        if not menuMusic.IsPlaying then
            return
        end

        if menuMusic.TimePosition >= MENU_LOOP_END then
            menuMusic.TimePosition = MENU_LOOP_START
        end
    end)
end

local function stopMenuLoop()
    if menuLoopConnection then
        menuLoopConnection:Disconnect()
        menuLoopConnection = nil
    end
end

local function forceHideMenu()
    mainFrame.Visible = false

    for _, obj in ipairs(mainFrame:GetDescendants()) do

```

```

        if obj:IsA("GuiObject") then
            obj.Visible = false
        end
    end

    if popups then
        popups.Visible = false

        for _, obj in ipairs(popups:GetDescendants()) do
            if obj:IsA("GuiObject") then
                obj.Visible = false
            end
        end
    end

    for _, obj in ipairs(screenGui:GetDescendants()) do
        if obj:IsA("GuiObject") then
            if not obj:IsDescendantOf(storyFrame) and obj ~= transition then
                obj.Visible = false
            end
        end
    end

    storyFrame.Visible = true
    transition.Visible = true
end

local function setupStoryUI()
    storyFrame.Visible = true

    if storyFrame:IsA("CanvasGroup") then
        storyFrame.GroupTransparency = 1
    end

    local storyBackground = storyFrame:FindFirstChild("Background")
    if storyBackground and storyBackground:IsA("Frame") then
        storyBackground.BackgroundTransparency = 0.65
        storyBackground.ZIndex = 50
    end

    if not dialogueBox then
        speakerLabel.AnchorPoint = Vector2.new(0.5, 0.5)
        speakerLabel.Position = UDim2.new(0.5, 0, 0.68, 0)
        speakerLabel.Size = UDim2.new(0.65, 0, 0.06, 0)
        speakerLabel.BackgroundTransparency = 1
        speakerLabel.TextXAlignment = Enum.TextXAlignment.Left
        speakerLabel.ZIndex = 52

        storyText.AnchorPoint = Vector2.new(0.5, 0.5)
    end
end

```

```

storyText.Position = UDim2.new(0.5, 0, 0.77, 0)
storyText.Size = UDim2.new(0.65, 0, 0.16, 0)
storyText.BackgroundTransparency = 1
storyText.TextXAlignment = Enum.TextXAlignment.Left
storyText.TextYAlignment = Enum.TextYAlignment.Top
storyText.TextWrapped = true
storyText.ZIndex = 52

hint.AnchorPoint = Vector2.new(0.5, 0.5)
hint.Position = UDim2.new(0.5, 0, 0.92, 0)
hint.Size = UDim2.new(0.5, 0, 0.04, 0)
hint.BackgroundTransparency = 1
hint.TextXAlignment = Enum.TextXAlignment.Center
hint.ZIndex = 52
else
    dialogueBox.Visible = true
    dialogueBox.AnchorPoint = Vector2.new(0.5, 0.5)
    dialogueBox.Position = UDim2.new(0.5, 0, 0.78, 0)
    dialogueBox.Size = UDim2.new(0.72, 0, 0.24, 0)
    dialogueBox.BackgroundColor3 = Color3.fromRGB(0, 0, 0)
    dialogueBox.BackgroundTransparency = 0.25
    dialogueBox.BorderSizePixel = 0
    dialogueBox.ZIndex = 52

    speakerLabel.AnchorPoint = Vector2.new(0, 0)
    speakerLabel.Position = UDim2.new(0.05, 0, 0.08, 0)
    speakerLabel.Size = UDim2.new(0.9, 0, 0.2, 0)
    speakerLabel.BackgroundTransparency = 1
    speakerLabel.TextXAlignment = Enum.TextXAlignment.Left
    speakerLabel.ZIndex = 53

    storyText.AnchorPoint = Vector2.new(0, 0)
    storyText.Position = UDim2.new(0.05, 0, 0.32, 0)
    storyText.Size = UDim2.new(0.9, 0, 0.42, 0)
    storyText.BackgroundTransparency = 1
    storyText.TextXAlignment = Enum.TextXAlignment.Left
    storyText.TextYAlignment = Enum.TextYAlignment.Top
    storyText.TextWrapped = true
    storyText.ZIndex = 53

    hint.AnchorPoint = Vector2.new(0.5, 0.5)
    hint.Position = UDim2.new(0.5, 0, 0.86, 0)
    hint.Size = UDim2.new(0.8, 0, 0.15, 0)
    hint.BackgroundTransparency = 1
    hint.TextXAlignment = Enum.TextXAlignment.Center
    hint.ZIndex = 53
end

speakerLabel.Text = ""

```

```

        storyText.Text = ""
        hint.Visible = false
    end

    local function setCameraTo(cameraName, instant)
        if not cameraName then
            return
        end

        local cameraPart = storyCameraFolder.FindFirstChild(cameraName)

        if not cameraPart or not cameraPart:IsA("BasePart") then
            warn("Camera part tidak ketemu: " .. cameraName)
            return
        end

        camera = workspace.CurrentCamera
        camera.CameraType = Enum.CameraType.Scriptable

        if instant then
            camera.CFrame = cameraPart.CFrame
        else
            tween(camera, {
                CFrame = cameraPart.CFrame
            }, 1.2)
        end
    end

    local function typeText(text, speed)
        storyText.Text = ""

        for _, last in utf8.graphemes(text) do
            storyText.Text = string.sub(text, 1, last)
            task.wait(speed or 0.035)
        end
    end

    local function waitForNext()
        hint.Visible = true

        local clicked = false
        local connection

        connection = UserInputService.InputBegan:Connect(function(input,
gameProcessed)
            if gameProcessed then
                return
            end

```

```

        if input.UserInputType == Enum.UserInputType.MouseButton1
            or input.UserInputType == Enum.UserInputType.Touch
            or input.KeyCode == Enum.KeyCode.Space
            or input.KeyCode == Enum.KeyCode.Return then

                clicked = true
            end
        end)

    repeat
        task.wait()
    until clicked

    if connection then
        connection:Disconnect()
    end

    hint.Visible = false
end

local function restoreGameplayCamera()
    player:SetAttribute("CameraPhase", "Gameplay")

    camera = workspace.CurrentCamera

    local character = player.Character or player.CharacterAdded:Wait()
    local humanoid = character:WaitForChild("Humanoid", 5)
    local root = character:WaitForChild("HumanoidRootPart", 5)

    if not humanoid or not root then
        warn("Humanoid atau HumanoidRootPart tidak ditemukan. Kamera
gameplay gagal dipulihkan.")
        return
    end

    root.Anchored = false

    humanoid.WalkSpeed = 16
    humanoid.JumpPower = 50
    humanoid.JumpHeight = 7.2
    humanoid.AutoRotate = true

    for _, part in ipairs(character:GetDescendants()) do
        if part:IsA("BasePart") then
            part.LocalTransparencyModifier = 0
        end
    end

    camera.CameraType = Enum.CameraType.Custom

```

```

camera.CameraSubject = humanoid

task.wait(0.1)

camera.CameraType = Enum.CameraType.Custom
camera.CameraSubject = humanoid

camera.CFrame = CFrame.lookAt(
    root.Position + Vector3.new(0, 5, 10),
    root.Position + Vector3.new(0, 2, 0)
)

for i = 1, 10 do
    task.wait()
    camera.CameraType = Enum.CameraType.Custom
    camera.CameraSubject = humanoid
end

print("Gameplay camera restored.")
print("Character:", character:GetFullName())
print("Root position:", root.Position)
print("CameraType:", camera.CameraType)
print("CameraSubject:", camera.CameraSubject)
end

local function startStory()
    if isRunning then
        return
    end

    isRunning = true
    player:SetAttribute("CameraPhase", "Story")

    stopMenuLoop()

    task.spawn(function()
        fadeSound(menuMusic, 0, 1, true)
    end)

    storyMusic.Volume = 0
    storyMusic.Looped = false
    storyMusic.TimePosition = STORY_START_TIME
    storyMusic:Play()

    task.spawn(function()
        fadeSound(storyMusic, STORY_VOLUME, 1.2, false)
    end)

    local cameraScript = screenGui.FindFirstChild("Camera")

```

```

if cameraScript and cameraScript:IsA("LocalScript") then
    cameraScript.Enabled = false
end

setCoreGuiEnabled(false)
setPlayerControl(false)

transition.Visible = true
transition.BackgroundColor3 = Color3.fromRGB(0, 0, 0)
transition.BackgroundTransparency = 1
transition.ZIndex = 100

tween(transition, {
    BackgroundTransparency = 0
}, 0.35)

forceHideMenu()

camera = workspace.CurrentCamera
camera.CameraType = Enum.CameraType.Scriptable
setCameraTo(STORY[1].Camera, true)

setupStoryUI()

if storyFrame:IsA("CanvasGroup") then
    tween(storyFrame, {
        GroupTransparency = 0
    }, 0.35)
end

tween(transition, {
    BackgroundTransparency = 1
}, 0.6)

for index, line in ipairs(STORY) do
    if index > 1 then
        setCameraTo(line.Camera, false)
    end

    speakerLabel.Text = line.Speaker
    typeText(line.Text, 0.035)
    waitForNext()
end

tween(transition, {
    BackgroundTransparency = 0
}, 0.4)

task.spawn(function()

```

```

        fadeSound(storyMusic, 0, 1, true)
    end)

    storyFrame.Visible = false

    restoreGameplayCamera()

    setPlayerControl(true)
    setCoreGuiEnabled(true)

    tween(transition, {
        BackgroundTransparency = 1
    }, 0.6)

    transition.Visible = false
    screenGui.Enabled = false

    isRunning = false

    game:GetService("ReplicatedStorage"):WaitForChild("SawitStartHUD"):Fire()
end

startMenuMusic()

playButton.MouseButton1Click:Connect(startStory)

```

## Penjelasan :

### 1. Inisialisasi & Setup Awal (Penyusunan Bahan)

Saat game pertama kali dimulai, skrip akan mengumpulkan semua "bahan" atau layanan yang dibutuhkan dari sistem Roblox:

- **Layanan Game:** Mengambil *Players*, *TweenService* (untuk animasi halus), *UserInputService* (membaca klik/tombol pemain), *StarterGui*, *SoundService* (audio), dan *RunService*.
- **UI & Kamera:** Mendeteksi elemen UI seperti *MainFrame* (menu utama), *StoryFrame* (bingkai dialog cerita), *Transition* (efek layar hitam), dan folder kamera cerita di *Workspace*.
- **Konfigurasi Audio:** Menyiapkan musik menu dan musik cerita, lengkap dengan batas waktu putaran lagu (*looping*) agar musik menu terus berulang di detik tertentu (MENU\_LOOP\_START ke MENU\_LOOP\_END).

### 2. Memutar Musik Menu (Saat Pemain di Main Menu)

Di baris paling bawah, skrip langsung menjalankan fungsi `startMenuMusic()`.

- Musik menu diputar dengan volume yang sudah ditentukan.
- Menggunakan `RunService.Heartbeat` untuk terus memantau posisi lagu. Jika lagu menyentuh detik ke-38 (MENU\_LOOP\_END), otomatis posisinya dikembalikan ke detik ke-7 (MENU\_LOOP\_START). Ini membuat musik menu berputar tanpa jeda secara mulus.

### 3. Pemain Menekan Tombol "Play"

Skrip mendeteksi klik dari playButton melalui *event* MouseButton1Click yang memicu fungsi utama, yaitu startStory(). Di sinilah alur cerita dimulai:

#### A. Mematikan Menu & Transisi Hitam

- Sistem menandai bahwa cerita sedang berjalan (isRunning = true).
- Musik menu dimatikan secara perlahan (*fade out*), dan musik cerita mulai diputar di menit/detik tertentu (STORY\_START\_TIME).
- Menonaktifkan pergerakan karakter (setPlayerControl(false)) dan menyembunyikan UI bawaan Roblox seperti chat dan daftar pemain (setCoreGuiEnabled(false)).
- Layar dibuat menjadi **hitam pekat** menggunakan objek transition untuk menutupi proses perpindahan kamera.
- Menghapus/menyembunyikan paksa seluruh tampilan menu utama (forceHideMenu()).

#### B. Menampilkan Cutscene & Dialog Berantai (STORY)

- Kamera pemain diubah menjadi Scriptable (bisa diatur penuh oleh skrip) dan langsung diarahkan ke posisi kamera cerita pertama (StoryCam1).
- Efek layar hitam perlahan memudar menjadi transparan, memperlihatkan pemandangan game.
- Skrip melakukan perulangan (*looping*) membaca tabel data STORY:
  1. Skrip mengubah nama pembicara (Speaker) menjadi "Narator".
  2. Teks cerita dimunculkan satu per satu seperti efek mesin ketik (typeText).
  3. Memunculkan petunjuk (Hint) agar pemain mengklik layar atau menekan Space/Enter.
  4. Skrip akan **berhenti/menunggu** (waitForNext()) sampai pemain memberikan input.
  5. Setelah diklik, jika baris cerita berikutnya menggunakan kamera berbeda, kamera akan bergerak secara halus (*tweening*) ke posisi kamera baru tersebut.

#### 4. Selesai Cerita & Masuk ke Gameplay (Pelepasan Kontrol)

Setelah baris cerita terakhir selesai dibaca:

- Layar kembali ditutup menjadi **hitam pekat** untuk menyembunyikan proses pemulihan karakter.
- Musik cerita dimatikan perlahan.
- Skrip memanggil fungsi restoreGameplayCamera():
  - Mengembalikan tipe kamera menjadi Custom dan mengunci fokus kamera kembali ke tubuh karakter pemain (Humanoid).
  - Memastikan tubuh pemain tidak terkunci (Anchored = false).
  - Mengembalikan kecepatan jalan dan kekuatan lompat pemain ke angka normal.
- Layar hitam (transition) perlahan memudar hingga hilang sepenuhnya.
- Seluruh sistem UI menu (screenGui.Enabled = false) dimatikan agar tidak mengganggu permainan.
- Terakhir, skrip mengirim sinyal jarak jauh (Fire) ke *RemoteEvent* bernama SawitStartHUD di *ReplicatedStorage*. Ini biasanya digunakan untuk memicu UI permainan utama (seperti petunjuk misi atau HUD kapak/alat panen sawit) agar muncul di layar pemain.

### Mekanisme Menjual Sawit :

```
-- =====
• -- Mekanisme Jual Sawit ke Muis
• -- Player mendekati Muis, tekan 'E' untuk menjual sawit
• -- PalmFruits dikurangi, Uang ditambah sesuai harga
• -- =====
•
• local Players = game:GetService("Players")
• local ReplicatedStorage = game:GetService("ReplicatedStorage")
•
• -- Konfigurasi
• local HARGA_PER_SAWIT = 10 -- Harga per buah sawit (ubah sesuai
  keinginan)
• local SELL_COOLDOWN = 1 -- Cooldown antar penjualan (detik)
•
• -- Referensi NPC Muis
• local muis = workspace:FindFirstChild("Muis")
• if not muis then
•   warn("NPC Muis tidak ditemukan di Workspace!")
•   return
• end
•
• -- Cari ProximityPrompt di Muis
• local prompt = muis:FindFirstChild("SellPrompt")
• if not prompt then
•   warn("ProximityPrompt 'SellPrompt' tidak ditemukan di Muis!")
•   return
• end
•
• -- RemoteEvent untuk feedback ke client
• local sellEvent = ReplicatedStorage:FindFirstChild("SellSawitEvent")
•
• -- Debounce table untuk mencegah spam penjualan
• local sellDebounce = {}
•
• -- Fungsi jual sawit
• local function onSellTriggered(player)
•   -- Cek cooldown
•   if sellDebounce[player.UserId] then
•     return
•   end
•   sellDebounce[player.UserId] = true
•   task.delay(SELL_COOLDOWN, function()
•     sellDebounce[player.UserId] = nil
•   end)
•
•   -- Cari leaderstats player
•   local leaderstats = player:FindFirstChild("leaderstats")
```

- if not leaderstats then
- return
- end
- 
- local sawit = leaderstats:FindFirstChild("Sawit")
- local uang = leaderstats:FindFirstChild("Uang")
- 
- if not sawit or not uang then
- return
- end
- 
- -- Hitung jumlah sawit yang dijual
- local sawitCount = sawit.Value
- 
- if sawitCount <= 0 then
- -- Tidak ada sawit untuk dijual, kirim feedback
- if sellEvent then
- sellEvent:FireClient(player, 0, 0, false)
- end
- return
- end
- 
- -- Hitung total uang dari penjualan
- local totalUang = sawitCount \* HARGA\_PER\_SAWIT
- 
- -- Update leaderboard
- sawit.Value = 0
- uang.Value = uang.Value + totalUang
- 
- -- Kirim feedback ke client
- if sellEvent then
- sellEvent:FireClient(player, sawitCount, totalUang, true)
- end
- 
- print(player.Name .. " menjual " .. sawitCount .. " sawit seharga " .. totalUang .. " uang!")
- end
- 
- -- Hubungkan ProximityPrompt dengan fungsi jual
- prompt.Triggered:Connect(onSellTriggered)
- 
- print("Mekanisme Jual Sawit ke Muis berhasil dimuat! Harga per sawit: " .. HARGA\_PER\_SAWIT)

## Penjelasan :

### 1. Validasi Keberadaan Objek (Pengecekan Awal)

Saat server pertama kali berjalan, skrip akan memastikan semua komponen penting di game sudah siap:

- **Cek NPC Muis:** Skrip mencari karakter bernama Muis di Workspace. Jika tidak ada, skrip akan memunculkan peringatan (*warning*) dan langsung berhenti berjalan (*return*).
- **Cek Tombol Interaksi:** Di dalam tubuh Muis, skrip mencari objek bernama SellPrompt (sebuah *ProximityPrompt* yang memunculkan tombol 'E' di layar pemain). Jika tidak ketemu, skrip juga akan berhenti.
- **Cek Jaringan Jembatan (RemoteEvent):** Skrip mencari SellSawitEvent di *ReplicatedStorage*. Event ini berfungsi untuk mengirim sinyal balik dari server ke perangkat pemain (*client*), misalnya untuk memunculkan efek suara atau notifikasi UI "Berhasil Menjual!".

## 2. Sistem Anti-Spam (*Debounce Table*)

Skrip ini menyiapkan sebuah tabel kosong bernama `sellDebounce = {}`.

Karena ini adalah skrip server yang menangani banyak pemain, pencegahan spam diatur menggunakan **User ID pemain**. Ketika pemain menekan tombol jual, ID mereka dimasukkan ke dalam daftar tunggu selama 1 detik (`SELL_COOLDOWN`). Jika mereka mencoba menekan tombol lagi sebelum 1 detik selesai, skrip akan mengabaikannya (*return*).

## 3. Proses Transaksi saat Tombol Ditekan (*onSellTriggered*)

Ketika seorang pemain mendekati Muis dan menekan/menahan tombol 'E', fungsi `onSellTriggered(player)` akan dipicu dan membawa data pemain tersebut. Alurnya adalah sebagai berikut:

### A. Validasi Isi Dompot & Kantong Pemain

1. Skrip memeriksa folder *leaderstats* di dalam data pemain (tempat menyimpan data skor/mata uang).
2. Di dalam *leaderstats*, skrip mencari dua data penting: **Sawit** (jumlah sawit yang dibawa) dan **Uang** (saldo saat ini).
3. Skrip mengecek jumlah sawit pemain (`sawit.Value`). Jika nilainya **0 atau kurang dari 0**, transaksi dibatalkan. Server akan mengirim sinyal ke *client* (`sellEvent:FireClient`) dengan status `false` agar *client* tahu pemain tidak punya sawit untuk dijual.

### B. Perhitungan Matematika & Update Data

Jika pemain terbukti membawa sawit (misalnya membawa 5 buah sawit):

1. **Hitung Pendapatan:** Total uang dihitung dengan rumus:

$$Total\ Uang = Jumlah\ Sawit \times Harga\ Per\ Sawit$$

(Contoh:  $5 \times 10 = 50$  Uang).

2. **Kosongkan Kantong:** Data sawit milik pemain langsung diatur ulang menjadi 0 (`sawit.Value = 0`).
3. **Isi Dompot:** Saldo uang pemain ditambah berdasarkan hasil hitungan tadi (`uang.Value = uang.Value + totalUang`).

### C. Feedback Visual / Notifikasi

Setelah data di server berhasil diperbarui, server mengirim sinyal balik ke *client* yang bersangkutan melalui `SellSawitEvent`, membawa informasi: jumlah sawit yang terjual, total uang yang didapat, dan status `true` (sukses). Sinyal ini biasanya ditangkap oleh skrip lokal pemain untuk memunculkan pesan di layar seperti *"Berhasil menjual 5 sawit seharga 50 uang!"*.

Terakhir, server akan mencetak log bukti transaksi di *Output* server untuk keperluan *debugging* developer.

## Data Manajement :

```
-- DataManager.lua
local DataStoreService = game:GetService("DataStoreService")
local Players = game:GetService("Players")

-- Membuat database unik untuk game SawitDev
local SawitDataStore = DataStoreService:GetDataStore("SawitDev_GlobalSave_V1")

local function SavePlayerData(player)
    local userId = player.UserId
    local leaderstats = player:FindFirstChild("leaderstats")

    if leaderstats then
        -- Menyusun data ke dalam table (Dictionary)
        local dataToSave = {
            Uang = leaderstats.Uang.Value,
            Sawit = leaderstats:FindFirstChild("Sawit") and leaderstats.Sawit.Value
or 0
        }

        -- Menggunakan pcall (protected call) untuk menangani error jaringan
        local success, err = pcall(function()
            SawitDataStore:SetAsync(tostring(userId), dataToSave)
        end)

        if success then
            print("Data " .. player.Name .. " berhasil disimpan ke Cloud.")
        else
            warn("Gagal menyimpan data " .. player.Name .. ": " .. err)
        end
    end
end

-- Saat Pemain Masuk
Players.PlayerAdded:Connect(function(player)
    local userId = player.UserId

    -- Menunggu folder leaderstats dibuat oleh skrip PlayerSetup
    local leaderstats = player:WaitForChild("leaderstats")

    -- Menunggu stat Sawit dibuat oleh PlayerSetup
    leaderstats:WaitForChild("Sawit")

    -- Mencoba mengambil data dari Cloud
    local data
    local success, err = pcall(function()
        data = SawitDataStore:GetAsync(tostring(userId))
    end)
```

```

end)

if success and data then
    -- Jika data ditemukan, terapkan ke variabel di game
    leaderstats.Uang.Value = data.Uang or 0

    -- Muat data Sawit
    local sawitStat = leaderstats.FindFirstChild("Sawit")
    if sawitStat then
        sawitStat.Value = data.Sawit or 0
    end
    print("Data lama " .. player.Name .. " berhasil dimuat.")
elseif not success then
    warn("Error saat mengambil data " .. player.Name .. ": " .. err)
else
    print(player.Name .. " adalah pemain baru. Menggunakan data awal.")
end
end)

-- Saat Pemain Keluar
Players.PlayerRemoving:Connect(function(player)
    SavePlayerData(player)
end)

-- Sistem Auto-Save (Setiap 5 menit / 300 detik)
task.spawn(function()
    while true do
        task.wait(300)
        for _, player in ipairs(Players:GetPlayers()) do
            SavePlayerData(player)
        end
        print("Auto-save masal berhasil dijalankan.")
    end
end)
end)

```

## Penjelasan :

### 1. Menghubungkan Game ke Cloud Roblox

Saat server game menyala, skrip ini langsung meminta akses ke layanan penyimpanan cloud milik Roblox menggunakan DataStoreService.

- Skrip membuat atau membuka sebuah "brankas" database unik bernama "SawitDev\_GlobalSave\_V1".
- Semua data pemain di game kamu akan disimpan di dalam brankas ini menggunakan **User ID** (angka unik permanen milik setiap akun Roblox) sebagai kunci gemboknya.

### 2. Proses Pemulihan Data (Saat Pemain Masuk / PlayerAdded)

Ketika ada pemain yang bergabung ke dalam game, skrip akan menjalankan alur berikut:

1. **Menunggu Kesiapan Karung Data (leaderstats):** Skrip dengan sabar menunggu (`WaitForChild`) sampai skrip lain (biasanya bernama *PlayerSetup*) selesai membuat folder `leaderstats` beserta isi di dalamnya (Uang dan Sawit).
2. **Mengambil Data dari Cloud (GetAsync):** Menggunakan fungsi `pcall` (fitur pelindung agar game tidak *crash* jika internet Roblox sedang gangguan), skrip mengambil data lama pemain berdasarkan User ID mereka.
3. **Penerapan Data:**
  - **Jika Data Ditemukan (Pemain Lama):** Angka Uang dan Sawit yang tersimpan di cloud langsung dimasukkan ke dalam nilai `leaderstats` pemain. Pemain bisa melanjutkan game dengan kekayaan mereka yang sebelumnya.
  - **Jika Data Kosong (Pemain Baru):** Skrip menganggapnya sebagai pemain baru dan membiarkan nilai Uang serta Sawit bernilai 0 (atau sesuai setelan awal game).

### 3. Proses Pengamanan Data (Fungsi `SavePlayerData`)

Fungsi ini adalah mesin utama untuk membungkus dan mengirim data ke cloud. Ketika fungsi ini dipanggil, alurnya adalah:

1. **Membungkus ke dalam Paket (Table/Dictionary):** Skrip melihat isi `leaderstats` pemain saat itu, lalu mencatat angka Uang dan Sawit ke dalam satu paket tabel bernama `dataToSave`.
2. **Mengirim ke Cloud (SetAsync):** Paket data tersebut dikirim ke cloud Roblox dan menimpa data lama yang ada di brankas. Proses ini juga dibungkus `pcall` untuk mengantisipasi kegagalan jaringan.

Fungsi pengamanan data ini dipicu oleh **dua kondisi** di bawah ini:

#### A. Kondisi 1: Pemain Keluar Game (`PlayerRemoving`)

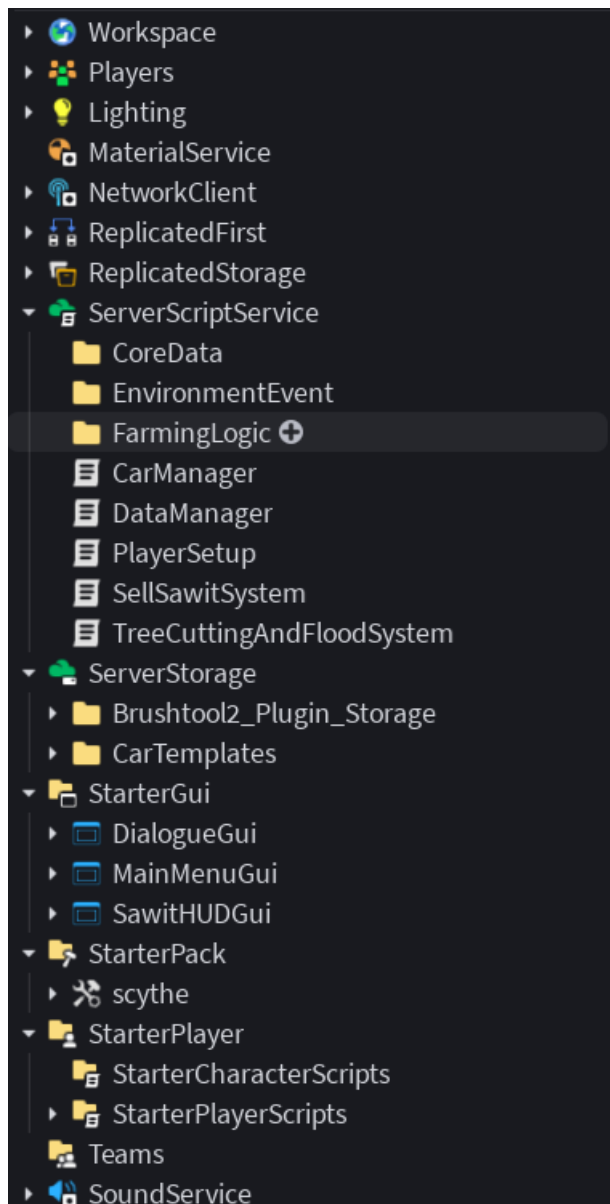
Ketika seorang pemain menekan tombol *Leave Game* atau memutuskan koneksi, skrip langsung menangkap momen tersebut dan memanggil `SavePlayerData(player)` untuk mengunci data terakhir mereka sebelum mereka benar-benar hilang dari server.

#### B. Kondisi 2: Sistem Kemanan Auto-Save (Setiap 5 Menit)

Untuk mengantisipasi hal-hal buruk seperti mati lampu, server game *crash/down*, atau pemain tiba-tiba terputus tanpa sempat memicu *PlayerRemoving*, skrip menjalankan sebuah fungsi latar belakang (*background thread*) menggunakan `task.spawn`.

- Fungsi ini berisi perulangan abadi (`while true do`) yang akan tidur/menunggu selama **300 detik (5 menit)**.
- Setelah 5 menit berlalu, skrip akan mengabsen seluruh pemain yang ada di dalam server saat itu menggunakan `Players:GetPlayers()`, lalu menyimpan data mereka satu per satu secara massal ke cloud. Proses ini berulang terus-menerus selama server aktif.

## Struktur Folder



Penjelasan :

### 1. Workspace (Dunia Fisika 3D Game)

Workspace adalah tempat di mana semua objek fisik yang bisa dilihat, disentuh, dan ditabrak oleh pemain berada. Jika suatu objek tidak ada di sini, pemain tidak akan bisa melihatnya di dalam game.

Karakter Pemain & NPC: Karakter kamu dan NPC seperti Muis diletakkan di sini agar bisa saling berinteraksi.

Folder Menu & StoryCameras: Folder berisi objek *Part* transparan yang berfungsi sebagai titik koordinat kamera saat *cutscene* cerita berlangsung.

Lingkungan Desa & Pabrik: Semua model pohon sawit, bangunan pabrik sawit, sungai, jalanan, dan tanah (terrain) diletakkan di bawah folder ini.

## 2. Players (Data Pemain Aktif)

Folder sistem yang otomatis mendata siapa saja pemain yang sedang terhubung ke dalam server game kamu saat ini. Saat pemain masuk, nama mereka akan muncul di folder ini. Di sinilah folder data seperti leaderstats (tempat menyimpan angka Uang dan Sawit) atau folder PlayerScripts dimasukkan secara otomatis oleh sistem saat game berjalan.

## 3. Lighting (Pencahayaan & Atmosfer)

Folder khusus untuk mengatur langit, cuaca, siang-malam, dan efek visual atmosfer game. Kamu bisa menambahkan efek seperti Sky (untuk mengganti tekstur langit), Blur (untuk efek buram saat menu utama), atau ColorCorrection agar suasana desa sawit terlihat lebih sinematik, gersang, atau berkabut.

## 4. MaterialService (Tekstur Material Kustom)

Layanan untuk mengatur tampilan fisik material di game (seperti aspal, kayu, tanah, besi). Kamu bisa memasukkan tekstur kustom generasi baru (PBR) di sini agar tekstur tanah perkebunan sawit atau dinding pabrik terlihat jauh lebih realistis dan detail.

## 5. NetworkClient & ReplicatedFirst

**NetworkClient:** Folder internal Roblox untuk mengatur koneksi internet antara komputer pemain dan server (tidak perlu kamu utak-atik).

**ReplicatedFirst:** Folder yang isinya akan diunduh **paling pertama** oleh HP/komputer pemain sebelum objek lainnya dimuat. Sangat cocok diletakkan skrip *Loading Screen* kustom agar pemain tidak melihat layar kosong saat game baru dibuka.

## 6. ReplicatedStorage (Gudang Bersama / Jembatan Data)

Gudang penyimpanan yang bisa diakses secara adil oleh Server maupun Client (Pemain). Di sinilah tempat terbaik menaruh objek RemoteEvent (seperti SellSawitEvent dan SawitStartHUD) yang berfungsi sebagai "surat" atau jembatan komunikasi agar skrip server bisa berbicara dengan skrip UI di layar pemain.

## 7. ServerScriptService (Jantung Logika Server)

Semua skrip di dalam folder ini adalah Script biasa (Server-Side) yang hanya bisa dibaca oleh server. Ini adalah tempat teraman untuk menaruh logika penting game agar tidak bisa dimanipulasi oleh *hacker*.

- **CoreData & EnvironmentEvent:** Manajemen data inti dan event perubahan lingkungan.
- **FarmingLogic:** Sistem pertumbuhan dan panen buah sawit.
- **CarManager:** Pengatur pemanggilan dan kendali mobil/truk sawit.
- **DataManager:** Sistem simpan-muat otomatis (*Save System*) kekayaan pemain ke Cloud.
- **PlayerSetup:** Penyiapan data awal pemain saat baru bergabung.
- **SellSawitSystem:** Logika transaksi jual-beli sawit menjadi uang.
- **TreeCuttingAndFloodSystem:** Mekanisme penebangan pohon dan pemicu bencana banjir.

## 8. ServerStorage (Gudang Rahasia Server)

Gudang penyimpanan aset yang hanya bisa dilihat oleh Server. Pemain sama sekali tidak bisa mengintip isi folder ini, sehingga sangat hemat memori dan aman.

**BrushTool2\_Plugin\_Storage:** Tempat penyimpanan vegetasi dari plugin pembuat peta.

**CarTemplates:** Cetakan/master data mobil. Mobil baru akan digandakan (*Clone*) dan dipindahkan ke Workspace hanya jika skrip server mengizinkannya.

## 9. StarterGui (Tampilan Layar / UI)

Folder template untuk seluruh antarmuka 2D di layar pemain. Semua isi folder ini akan otomatis dicopy ke dalam perangkat masing-masing pemain begitu mereka *spawn*.

**DialogueGui:** UI untuk teks cerita/narator pembuka.

**MainMenuGui:** UI untuk tombol klik *Play* di awal game.

**SawitHUDGui:** UI utama saat bermain (indikator uang, jumlah isi kantong sawit, dll).

## 10. StarterPack (Tas Peralatan Awal)

Gudang senjata atau peralatan. Setiap objek bertipe Tool yang ditaruh di sini akan langsung otomatis masuk ke dalam *inventory* / tas pemain (bisa diakses lewat tombol angka 1, 2, 3 dst di keyboard).

scythe: Alat dodos/egrek sawit yang langsung dipegang pemain sejak awal permainan tanpa harus membeli dulu.

## 11. StarterPlayer (Skrip & Pengaturan Karakter)

Folder khusus yang mengatur sifat dasar dari manusia/karakter di game kamu serta tempat menyimpan skrip lokal (LocalScript).

**StarterCharacterScripts:** Skrip yang berjalan di tubuh karakter dan akan di-*reset* ulang setiap kali pemain mati lalu hidup lagi.

**StarterPlayerScripts:** Skrip lokal yang berjalan di balik layar komputer pemain sejak awal masuk dan tidak akan ter-reset meskipun karakternya mati berulang kali (contohnya: skrip kamera cerita di Main Menu).

## 12. Teams (Sistem Kubu/Kelompok)

Folder opsional untuk membagi pemain ke dalam kelompok tertentu. Jika kamu ingin membagi permainan menjadi tim "Buruh Tani" dan tim "Bos Pabrik", kamu tinggal menambahkan objek Team di dalam folder ini.

## 13. SoundService (Pusat Efek Suara & Musik)

Gudang audio untuk mengelola semua suara di dalam game. Di sinilah tempat kamu menaruh folder MenuSounds yang berisi lagu MenuMusic dan StoryMusic. Suara yang ditaruh di sini bisa diatur volumenya secara global atau diputar langsung ke telinga pemain lewat skrip tanpa terpengaruh oleh jarak fisik karakter di dalam game